

Software Testing

**Testing is not a final phase.
It is a design, feedback and confidence mechanism.**

Course block

Write small tests, run them automatically, measure coverage, and use tests to support refactoring and AI-assisted changes.

Pedro García López · TAP

Why test early?

Most defects are cheaper to fix when discovered close to their origin.



- Errors are often introduced before coding starts.
- Late defects are harder to diagnose and more expensive to fix.
- Testing should start with requirements, design and examples.

Takeaway

Testing is not only “running the program”. It is a systematic way to discover mismatches between intent, design and behavior.

Error, fault, failure

Three related but different concepts.



```
// Fault: boundary condition is wrong
boolean isAdult(int age) {
    return age > 18;    // should be >= 18
}

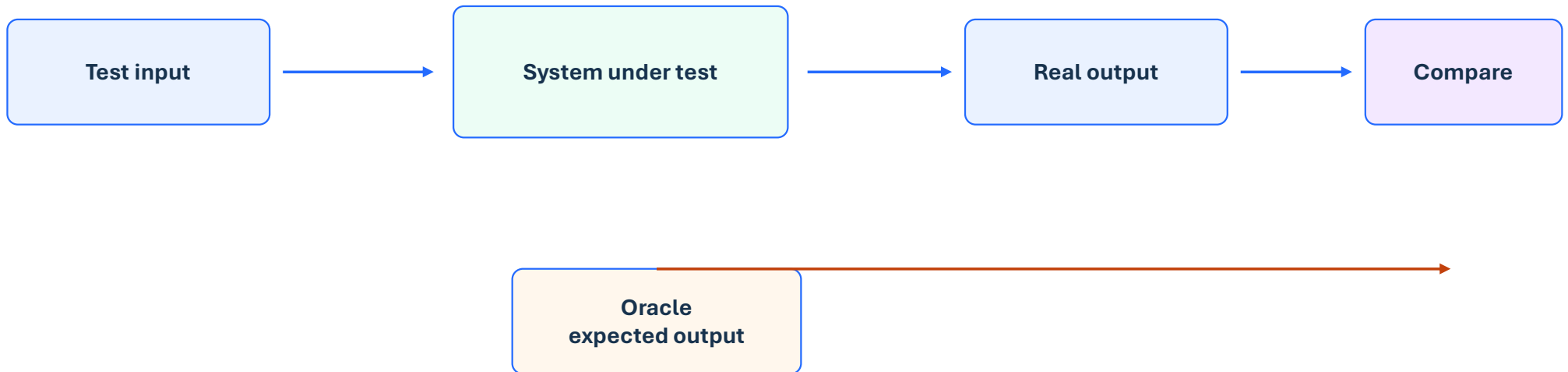
// Failure appears when age == 18
```

Verification vs validation

Verification: have we built the system right?
Validation: have we built the right system?

What is the testing process?

Input + expected behavior + execution + comparison.



- A test is useful only if it can decide pass/fail.
- The oracle may be a specification, example, property, snapshot or human review.
- Automated tests make this comparison repeatable.

Testing levels

Different scopes detect different kinds of problems.

Unit

One class/function. Fast, isolated, many tests.
Example: PricingPolicyTest.

Integration

Several components together.
Example: repository + database.

System

Whole application behavior.
Example: CLI/API use case.

Acceptance

User/business criteria.
Example: professor accepts grading workflow.

Fast feedback → broader confidence → deployment decision

Black-box and white-box testing

Where do we get test cases from?

Black-box / specification-based

Tests derive from the visible behavior.

Inputs, outputs, edge cases, business rules, examples, acceptance criteria.

Good for: validation and user-visible behavior.

White-box / structure-based

Tests derive from internal code structure.

Branches, loops, paths, conditions, data-flow, exceptions.

Good for: discovering untested implementation paths.

```
// Black-box case: grade = 4.9 -> fail  
// White-box case: cover both branches
```

Coverage-based testing

Coverage measures which parts of the code were exercised.

Statement coverage

Did each executable statement run?

Branch coverage

Did each decision outcome run?
if/else, switch cases.

Path coverage

Did each possible path run? Usually
infeasible for non-trivial programs.

```
int fee(int days) {  
    if (days <= 0) throw new  
    IllegalArgumentException();  
    if (days <= 2) return 10;  
    return 10 + (days - 2) * 4;  
}
```

Important

100% coverage does not prove correctness.
It only says which code was executed by the tests.

Fault-based and regression testing

Good tests are designed to catch likely faults.

Fault-based testing

Design tests from expected fault patterns: off-by-one errors, nulls, wrong operators, wrong comparison, missing cases.

```
// Bug report: students with exactly 5.0 are marked  
FAIL  
@Test  
void gradeFiveIsPass() {  
    assertEquals(PASS, classifier.classify(5.0));  
}
```

Regression testing

When a bug is found, add a test that fails before the fix and passes after it. Keep it forever.

Rule

Every important bug should become a regression test. Otherwise it can return silently.

What is a unit test?

A small automated test for a small piece of behavior.

- Tests one class or method from the programmer perspective.
- Runs fast and deterministically.
- Has a clear system under test.
- Avoids network, real databases, time, randomness when possible.

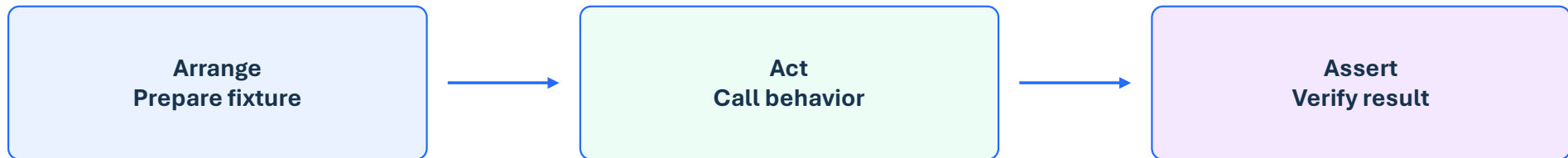
```
class PriceCalculatorTest {  
    @Test  
    void appliesDiscount() { ... }  
}
```

Vocabulary

SUT: system under test
CUT: class under test
MUT: method under test
Fixture: objects needed by tests
Runner: tool executing tests

Anatomy of a test method

Arrange · Act · Assert.



```
@Test
void totalPriceIncludesVat() {
    // Arrange
    var invoice = new Invoice(Money.euros(100));

    // Act
    Money total = invoice.totalWithVat(0.21);

    // Assert
    assertEquals(Money.euros(121), total);
}
```

JUnit 5 basic structure

Tests are ordinary Java methods with testing annotations.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class StudentGradeTest {

    @Test
    void studentPassesWithGradeFive() {
        Grade grade = new Grade(5.0);

        assertTrue(grade.isPass());
    }
}
```

Key annotations

- @Test
- @DisplayName
- @BeforeEach / @AfterEach
- @BeforeAll / @AfterAll
- @ParameterizedTest

Naming rule

Use test names that describe behavior, not implementation.

Assertions and exceptions

A test should clearly state what must be true.

```
assertEquals(42, result);
assertTrue(user.isActive());
assertFalse(order.isEmpty());
assertNotNull(report);
assertIterableEquals(expected, actual);

assertThrows(IllegalArgumentException.class, () -> {
    new Grade(-1);
});
```

Good assertion

Specific failure message, one clear reason to fail, verifies observable behavior.

Weak assertion

Only checks that code runs, tests private details, or repeats production logic inside the test.

```
// Bad: no real oracle
assertDoesNotThrow(() -> service.process(input));
```

Fixtures: setup and teardown

Avoid duplicated setup while keeping tests independent.

```
class ReportServiceTest {  
    private ReportService service;  
    private FakeRepository repo;  
  
    @BeforeEach  
    void setUp() {  
        repo = new FakeRepository();  
        service = new ReportService(repo);  
    }  
  
    @Test  
    void findsExistingReport() { ... }  
}
```

Independence

Each test should be able to run alone.
No hidden order dependency.
No shared mutable state unless reset.

Prefer

Small fixture, explicit data, helper methods or test data builders for complex objects.

Parameterized tests

One behavior, many examples.

```
@ParameterizedTest
@CsvSource({
    "0, FAIL",
    "4.9, FAIL",
    "5.0, PASS",
    "10.0, PASS"
})
void classifiesGrade(double value, Result expected) {
    assertEquals(expected, classifier.classify(value));
}
```

Why useful?

Boundary values become visible.
Duplicate test code disappears.
The test reads like a small specification table.

Typical inputs

Nulls, empty strings, zero, one, negative numbers,
maximum values, invalid formats, duplicate
elements.

Test doubles

Replace difficult dependencies with controlled collaborators.

Dummy

Passed around but not used.

Stub

Returns predefined answers.

Fake

Working simple implementation.

Mock

Verifies interactions.

Spy

Records calls for later checks.

```
interface EmailSender {
    void send(String to, String body);
}

class FakeEmailSender implements EmailSender {
    List<String> sent = new ArrayList<>();
    public void send(String to, String body) {
        sent.add(to + ":" + body);
    }
}
```

Design for testability

Dependency injection makes unit tests possible.

```
// Hard to test: creates real dependency
class NotificationService {
    private final SmtpClient smtp = new SmtpClient();
}

// Easier to test: dependency is injected
class NotificationService {
    private final EmailSender sender;

    NotificationService(EmailSender sender) {
        this.sender = sender;
    }
}
```

Benefit

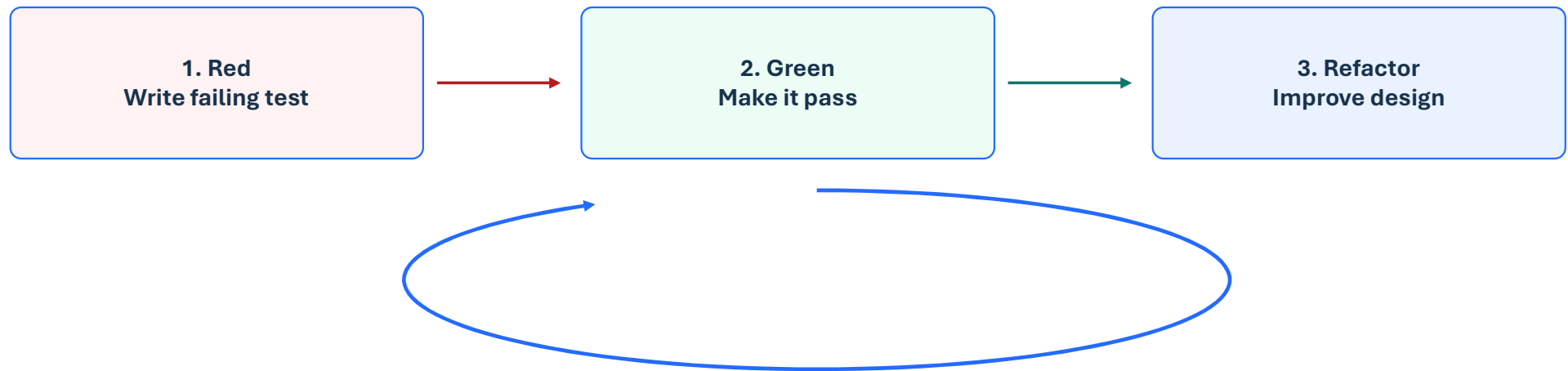
The production code can use `SmtpEmailSender`; the test can use `FakeEmailSender`. The service behavior is tested without sending real emails.

Connection to patterns

Strategy, Adapter, Factory and Dependency Injection all improve testability by depending on abstractions.

Test-Driven Development

Red · Green · Refactor.



- Add a test for the next small behavior.
- Run all tests and see the new one fail.
- Write the simplest code that passes.
- Refactor safely while tests stay green.

TDD example: small specification

Start from behavior, not from implementation.

Feature

A GradeClassifier returns PASS when the grade is at least 5.0 and FAIL otherwise.

First example

Given grade 5.0
When classifying it
Then the result is PASS

```
@Test
void gradeFiveIsPass() {
    var classifier = new GradeClassifier();

    Result result = classifier.classify(5.0);

    assertEquals(Result.PASS, result);
}
```

TDD step 1: the test fails

The failure is useful: it proves the test is connected to missing behavior.

```
class GradeClassifierTest {  
    @Test  
    void gradeFiveIsPass() {  
        var classifier = new GradeClassifier();  
        assertEquals(PASS, classifier.classify(5.0));  
    }  
}  
  
// Compilation error or failed assertion:  
// GradeClassifier does not exist / returns wrong value
```

Red means progress

The test describes a behavior that the system does not yet implement.

Do not skip red

If the test passes immediately, maybe it does not test the intended behavior.

TDD step 2: make it pass

Write only enough production code for the test.

```
enum Result { PASS, FAIL }

class GradeClassifier {
    Result classify(double grade) {
        return Result.PASS; // minimal code for first test
    }
}
```

This looks too simple

That is acceptable in TDD. The next test forces the design to grow.

```
@Test
void gradeBelowFiveIsFail() {
    var classifier = new GradeClassifier();

    assertEquals(FAIL, classifier.classify(4.9));
}
```

Second test

Now the fake implementation is no longer enough.

TDD step 3: refactor

When tests are green, improve the design.

```
class GradeClassifier {
    Result classify(double grade) {
        if (grade < 0 || grade > 10) {
            throw new IllegalArgumentException("grade");
        }
        return grade >= 5.0 ? PASS : FAIL;
    }
}
```

Refactoring checkpoint

Behavior should not change. Tests should remain green. Names, structure, duplication and responsibilities improve.

TDD design effect

Tests push APIs to be small, explicit and easy to use.

```
// More examples can now be parameterized
@CsvSource({"0,FAIL", "4.9,FAIL", "5,PASS"})
```

TDD principles

The method is small steps plus constant feedback.

DRY

Do not repeat yourself. Remove duplication in code and tests.

KISS

Keep it simple. Implement the simplest design that works.

YAGNI

Do not implement future behavior before it is needed.

SRP

One class should have one main reason to change.

- TDD is more than testing: it is a design technique.
- The tests document behavior.
- Refactoring becomes safe because behavior is protected.

Unit test quality

Tests can also be bad code.

Good tests

- Fast
- Deterministic
- Readable
- Independent
- Behavior-oriented
- Small failure messages

Bad tests

- Flaky
- Slow
- Depend on test order
- Assert implementation details
- Use sleeps
- Test too much at once

```
// Smell: testing implementation details
assertEquals(3, service.getInternalCache().size());

// Better: test visible behavior
assertEquals(report, service.findReport(id));
```

Code coverage

Coverage is a feedback signal, not a quality guarantee.



Metrics

- Instruction coverage
- Line coverage
- Branch coverage
- Method/class coverage

Interpretation

Low coverage: probably risky.
High coverage: still inspect assertions, edge cases and behavior.

JaCoCo with Maven

A common Java coverage tool.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.13</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

Run locally

mvn test

Then open:
<target/site/jacoco/index.html>

Teaching use

Ask students to explain uncovered branches and whether they are meaningful.

Coverage thresholds

Use thresholds carefully: they protect against test erosion.

```
<execution>
  <id>check</id>
  <goals><goal>check</goal></goals>
  <configuration>
    <rules>
      <rule>
        <element>BUNDLE</element>
        <limits>
          <limit>
            <counter>BRANCH</counter>
            <value>COVEREDRATIO</value>
            <minimum>0.70</minimum>
          </limit>
        </limits>
      </rule>
    </rules>
  </configuration>
</execution>
```

Caution

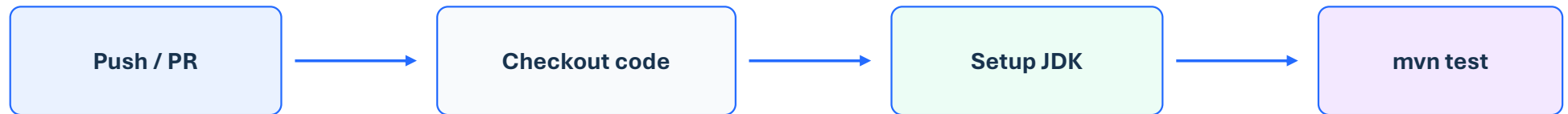
A threshold can be gamed with useless tests.
Review quality, not only percentages.

Useful policy

Start modestly, raise gradually, and require new code to be covered by meaningful tests.

GitHub Actions: continuous testing

Run the test suite automatically on every push and pull request.



- Tests become part of the development workflow.
- A failing test can block merging.
- The same commands run on every machine and branch.

GitHub Actions workflow for Java

Minimal Maven workflow.

```
name: Java CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: mvn -B test
```

Where it lives

.github/workflows/java-ci.yml

What students learn

Build reproducibility, automation, branch checks and early feedback.

GitHub Actions + coverage reports

Generate coverage and keep the report as a workflow artifact.

```
- name: Run tests with coverage
  run: mvn -B test jacoco:report

- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: target/site/jacoco/
```

```
mvn test jacoco:report
# produces: target/site/jacoco/index.html
```

Optional integrations

Codecov, Coveralls, SonarCloud or GitHub Pages can publish coverage and quality dashboards.

Assessment idea

Ask students to attach CI status and coverage report to their practical assignment submission.

Test reports in CI

CI should make failures easy to diagnose.

Console log

Fast but hard to navigate for large projects.

JUnit XML

Standard test result format consumed by many CI tools.

Coverage HTML

Visual report: lines, branches, classes and packages.

```
target/surefire-reports/  
├── TEST-com.example.GradeTest.xml  
└── com.example.GradeTest.txt  
  
target/site/jacoco/  
└── index.html
```

AI-assisted coding needs tests

AI can generate code quickly; tests define acceptable behavior.

Without tests

AI may change behavior silently.
Refactoring is risky.
Reviewers inspect too much manually.

With tests

AI changes are checked automatically.
Regression tests protect old behavior.
Coverage highlights untouched paths.

```
Prompt for AI agent:  
1. Add support for --format json.  
2. First add failing tests.  
3. Do not modify PdfExporter.  
4. Run mvn test and show results.
```

Spec-driven testing

Specifications should produce tests before implementation.

```
## Requirement
The CLI shall export reports in JSON format.
```

```
## Acceptance criteria
- --format json produces valid JSON.
- Unknown formats return exit code 2.
- Existing PDF behavior remains unchanged.
```

```
## Tests
- jsonExportProducesValidJson()
- unknownFormatReturnsError()
- pdfExportStillWorks()
```

Connection

Spec → tests → implementation → CI.

The spec gives the test oracle. The tests protect the spec during future changes.

OpenSpec-style workflow

proposal.md
design.md
tasks.md
contract tests

Testing assignment proposal

A practical assignment that combines unit tests, TDD and CI.

Task

Build a small report-processing module with CSV input and JSON/PDF output.

Required tests

Unit tests for parsers and exporters.
Parameterized boundary tests.
At least one regression test for a discovered bug.

Required automation

GitHub Actions workflow.
JaCoCo coverage report.
README badge or screenshot of passing CI.

Reflection

Explain one design change made possible by tests and one AI-generated change you reviewed.

Review checklist for student projects

Use this checklist before submitting.

- Can I run all tests with one command?
- Do tests fail if I intentionally break the code?
- Are edge cases covered?
- Is CI green on GitHub?
- Is coverage report generated?
- Are tests readable enough to act as documentation?

```
# Local commands
mvn test
mvn test jacoco:report

# CI file
.github/workflows/java-ci.yml
```

Minimal quality bar

A project without automated tests is not ready for refactoring, AI-assisted extension, or serious review.

Common mistakes

Testing anti-patterns to avoid.

Testing implementation

Asserting private fields, internal cache sizes, or exact call order without need.

Too much mocking

Mocks everywhere make tests fragile and disconnected from behavior.

Flaky tests

Depend on timing, real network, shared files or test order.

No assertions

The test only executes code but does not verify behavior.

Coverage obsession

Raising numbers without improving test quality.

Slow unit tests

Unit tests should not require full deployment or real cloud services.

Summary

Unit tests turn design intent into executable feedback.

Remember

Testing starts early.
Unit tests are small and automated.
TDD is Red-Green-Refactor.
Coverage measures execution, not correctness.

Modern workflow

Write tests.
Run locally.
Measure coverage.
Run in GitHub Actions.
Use tests to control AI-assisted changes.

Good tests reduce fear: they make change safer, review clearer, and refactoring practical.

References and source material

Use these to deepen the topic.

- Original TAP testing slides: testing process, TDD steps, unit-test terminology, coverage and inspections.
- JUnit 5 User Guide: assertions, lifecycle, parameterized tests.
- JaCoCo Maven Plugin: coverage report and check goals.
- GitHub Actions documentation: Java workflow, setup-java, upload-artifact.
- Fowler: Refactoring and test-supported evolutionary design.

```
Course repository idea:  
Add .github/workflows/java-ci.yml  
Add pom.xml JaCoCo configuration  
Run tests and coverage on every PR
```